Universidad de Málaga

Escuela Técnica de Ingeniería de Telecomunicación

Procesado y Análisis de Datos con Python

Autores:

Miguel Estrada Grijalba y Ana María López Lupiáñez

27 de abril de 2026

Índice

1. Introducción **2**

2. Fase 1 **2**

2.1. Lectura en tiempo real 2

2.2. Identificación de mensajes DL-SDAP en el log 3

2.3. Almacenamiento y agregación por segundo en un Dataframe 4

2.4. Representación en tiempo real con Dash 6

2.5. Almacenamiento en archivo Parquet 8

3. Fase 2 **10**

3.1. Identificación de PLMN, PCI y RNTI de cada UE 10

3.1.1. Fundamentos de los Parámetros de Red 10

3.1.2. Integración en el sistema y justificación 11

4. Main **12**

5. Funcionamiento del programa **13**

6. Conclusiones **14**

A. Código Fuente del Proyecto **16**

A.1. Script principal de monitorización (log-parser.py) 16

A.2. Leer Parquet (leer_parquet.py) 20

1. Introducción

En el contexto actual de las telecomunicaciones, analizar el tráfico de las redes 5G en tiempo real se ha convertido en algo esencial para entender cómo están funcionando y qué rendimiento ofrecen. Este proyecto cumple precisamente con esa idea: desarrollar una herramienta en *Python* capaz de monitorizar de forma continua los datos generados por una red móvil, procesarlos de manera eficiente y mostrarlos en una interfaz gráfica clara y fácil de interpretar.

El trabajo se ha dividido en dos grandes fases. En la primera, el sistema se encarga de leer los registros (*logs*) de la red de forma continua y asíncrona, lo que permite procesar la información sin que el programa se bloquee. En este proceso, el código identifica los mensajes relacionados con la descarga de datos (DL-SDAP) y calcula cuántos *bytes* recibe cada usuario por segundo. Toda esta información se envía a una página web interactiva desarrollada con *Dash*, donde una gráfica se actualiza automáticamente cada segundo para mostrar la evolución del tráfico en tiempo real. Además, para evitar la pérdida de datos, el sistema realiza copias de seguridad automáticas en formato *Parquet* cada 30 segundos.

En la segunda fase, se amplía la funcionalidad del sistema incorporando información más detallada sobre la conexión de cada usuario. Para ello, se extraen parámetros clave de la red 5G, como el identificador del operador (PLMN), la celda física que da servicio al usuario (PCI) y su identificador temporal (RNTI). Todos estos datos se integran en una tabla dentro de la misma interfaz web, lo que permite tener una visión más completa y contextualizada: no solo cuánto tráfico se está descargando, sino también en qué condiciones y desde qué punto de la red se está produciendo.

2. Fase 1

2.1. Lectura en tiempo real

Para permitir que el escript procese un flujo continuo de datos en tiempo real, como el tráfico de una red 5G, se ha implementado la función asíncrona `tail_file_producer`. Esta función espera inicialmente a que el archivo exista y, una vez disponible, posiciona el puntero de lectura al final mediante la instrucción `f.seek(0,2)`, con el objetivo de ignorar los datos previamente almacenados.

A partir de ese momento, se ejecuta un bucle infinito que intenta leer nuevas líneas del archivo. En caso de no detectarse contenido nuevo, el proceso introduce una breve pausa `(await asyncio.sleep(0.1))` antes de reintentar la lectura. Este mecanismo evita un uso innecesario de la CPU debido a comprobaciones constantes, permitiendo así una monitorización eficiente en tiempo real sin interrumpir la ejecución.

Asimismo, resulta fundamental encapsular esta lógica dentro de una tarea asíncrona

basada en `asyncio`, con el fin de evitar el bloqueo del resto de funcionalidades del sistema y garantizar que la interfaz web permanezca operativa y fluida.

Las líneas de código claves para la ejecución de esta parte son:

```
async def tail_file_producer(path: str, data_queue: asyncio.Queue):
    file = Path(path)

    # 1. Espera a que el simulador cree el archivo
    while not file.exists():
    await asyncio.sleep(0.5)

    with open(path, "r") as f:
    # 2. Se posiciona al final del archivo
    f.seek(0, 2)

    # 3. Bucle infinito de lectura
    while True:
        line = f.readline()

        if line:
            #....
        else:
            # 4. Espera activa si no hay líneas nuevas
            await asyncio.sleep(0.1)
```

Código 1: Lectura en tiempo real.

2.2. Identificación de mensajes DL-SDAP en el log

Para la identificación de mensajes relevantes dentro del archivo de log, se emplea la librería `re`, mediante la cual se define un patrón de búsqueda que actúa como plantilla sobre el texto no estructurado. Cada vez que se lee una nueva línea, la función `search()` evalúa si esta contiene las cadenas clave *[SDAP]* y *DL: TX PDU*. En caso afirmativo, se utilizan grupos de captura *(\d+)* para extraer de manera precisa los valores numéricos asociados tanto al identificador de usuario (*ue=...*) como al tamaño del paquete transmitido (*pdu_len=...*).

```
patron_sdap = re.compile(r"\[SDAP\s+\].*ue=(\d+).*DL: TX PDU.*pdu_len=(\
    d+)")
```

Código 2: Definición del patrón.

```
if line:
    match_sdap = patron_sdap.search(line)
    if match_sdap:
        ue_id = int(match_sdap.group(1))
```

```
        bytes_len = int(match_sdap.group(2))
```

Código 3: Búsqueda de los datos DL-SDAP.

El patrón definido garantiza un filtrado preciso, al exigir la presencia de la cadena *DL: TX PDU*, correspondiente a transmisiones en downlink. De este modo, se descartan automáticamente aquellos mensajes relacionados con tráfico de subida (UL) o con señalización. La utilización de expresiones regulares resulta adecuada en este contexto, dado que los archivos de log contienen grandes volúmenes de información correspondientes a distintos protocolos de red (como RRC o MAC). Frente a métodos tradicionales de procesamiento de cadenas como `split()` o `find()`, la compilación mediante un patrón Regex (`re.compile`) permite optimizar la búsqueda en memoria, al escanear miles de líneas por segundo sin degradar el rendimiento del programa.

2.3. Almacenamiento y agregación por segundo en un Dataframe

La estructura del programa se basa en un modelo **Productor-Consumidor**. Se trata de una arquitectura en la que se consigue desacoplar la parte que captura los datos (Productor) de la parte que los procesa (Consumidor), permitiendo que ambos operen a ritmos diferentes y de forma asíncrona.

Tras haber definido mediante expresiones regulares, en el apartado anterior el tipo de mensajes que extraeremos del archivo *log*, podemos estudiar cómo es la comunicación y el almacenamiento entre los dos procesos para lograr los objetivos.

En este diseño, el *DataFrame* actúa como el almacén central y la frontera de desacoplamiento del sistema. Es la parada técnica obligatoria donde los datos quedan ordenados y estructurados, justo antes de que la herramienta de interfaz (Dash) los lea para dibujar las gráficas.

El productor, como su nombre indica es el encargado de producir la información, lee en tiempo real los datos que le van llegando del *log* y extrae los mensajes deseadaos, posteriormente llega la parte del código que interesa en este apartado:

```
async def tail_file_producer(path: str, data_queue: asyncio.Queue):
    .
    .
    .
    t_actual = time.time()
    if t_actual - t_inicio >= 1.0:
        vector = tuple(sumas_segundo)
        wait data_queue.put(vector)
        sumas_segundo = [0] * n_ues
        t_inicio = t_actual
```

Código 4: Lógica de agregación y envío de datos cada segundo (Productor).

En el Código 4, podemos observar la sección final del productor y sus líneas más relevantes con respecto a la función que desempeña. El programa comienza comparando el tiempo actual con el tiempo de inicio de la última iteración en la que se enviaron datos al consumidor. Si esta diferencia es mayor o igual a 1 segundo, se accede al bloque condicional (`if`), cumpliendo así con el requisito de la práctica de realizar la agregación de datos cada segundo.

Tras verificar que se cumple la condición temporal, el vector de agregados `sumas_segundo` se convierte en una tupla (`tuple`) para garantizar que los datos se mantienen intactos durante su transmisión. Posteriormente, mediante la instrucción `await data_queue.put(vector)`, se introduce la tupla en la cola de comunicación asíncrona con el consumidor. Por último, se restablece a cero el vector donde se guardan los datos acumulados y se reinicia el contador de tiempo para el siguiente ciclo de captura.

Por otro lado, el consumidor tiene el siguiente código:

```
1  async def consumer(data_queue: asyncio.Queue):
2      global agg_df, vector_list
3
4      while True:
5          vector_tuple = await data_queue.get()
6          vector_list = list(vector_tuple)
7          new_row = pl.DataFrame([vector_list], schema=agg_df.columns,
       orient="row")
8          agg_df = pl.concat([agg_df, new_row], how="vertical")
9
10         data_queue.task_done()
```

Código 5: Captura de datos (Consumidor).

La función del consumidor espera constantemente la llegada de información a través de la cola. En la primera línea dentro del bucle `while` (`vector_tuple=awaitdata_queue .get()`), dado que la información se recibe en formato de tupla, esta se debe convertir a una lista para poder leerla y transferirla al *dataframe*.

Para integrar los datos recién recibidos en el registro histórico, es necesario adaptar su formato a los requisitos de la librería Polars. En primer lugar, la lista de valores (`vector_list`) se transforma en un *DataFrame* temporal de una sola fila (`new_row`). Para lograrlo, la lista se encapsula entre corchetes adicionales, se hereda la estructura de columnas exacta del registro principal (`schema=agg_df.columns`) y se define explícitamente su orientación horizontal (`orient="row"`).

Esta asignación del esquema es un paso fundamental, ya que una lista de números por sí sola carece de identificadores. Al utilizar `agg_df.columns`, se garantiza que la nueva fila adquiera exactamente los mismos títulos o cabeceras que la tabla global. De esta forma, se asegura la compatibilidad total entre ambas estructuras, requisito indispensable para que la posterior operación de concatenación vertical sea exitosa y no produzca errores por

nombres de columna discordantes.

Finalmente, una vez formateada la nueva fila, se procede a su inserción mediante la función `pl.concat`. Al especificar el parámetro `how="vertical"`, se consigue que los nuevos datos se apilen de forma secuencial en la parte inferior de la tabla (`agg_df`), generando un registro histórico continuo que será consumido por la interfaz gráfica.

Por último, la instrucción `data_queue.task_done()` sirve simplemente para avisar a la cola de que se ha terminado de procesar el último dato extraído.

La librería `asyncio` lleva una cuenta interna de las tareas que quedan pendientes. Al usar la función `get()`, se saca un elemento de la cola, pero es al llamar a `task_done()` cuando se confirma definitivamente que el trabajo con ese dato ha concluido.

2.4. Representación en tiempo real con Dash

Dash es un *framework* de Python que sirve para crear distintas aplicaciones de manera sencilla. Permite conectar el *backend* con una interfaz gráfica simple, facilitando al usuario una visualización rápida y fácil de los resultados.

La estructura visual de la aplicación se define en el bloque `app.layout`, el cual actúa como el esqueleto de la página web agrupando los diferentes componentes dentro de un contenedor principal (`html.Div`). En este código, la interfaz está compuesta por los siguientes elementos:

- Un título descriptivo (`html.H3`).
- Una tabla (`dag.AgGrid`) configurada para mostrar las propiedades de los usuarios (PLMN, PCI, RNTI).
- Un espacio reservado para el gráfico de líneas (`dcc.Graph`).
- Un componente invisible (`dcc.Interval`) que funciona como un cronómetro interno. Está configurado a 1000 milisegundos (1 segundo) y se encarga de marcar el ritmo de actualización de la página.

```
app.layout = html.Div([
    html.H3("Monitorización 5G y Parámetros de Red (Fase 1 y 2)"),
    #  Grid Transpuesto para mostrar las propiedades limpiamente ---
    dag.AgGrid(
        id='live-adgrid',
        style={"height": 200, "width": "100%"}, # Altura aumentada
        columnSize='autoSize',
        columnDefs=[
            {"field": "UE", "headerName": "ID Usuario"},
            {"field": "Bytes", "headerName": "Volumen (Último Seg)", "
    type": "rightAligned"},
```

```
            {"field": "PLMN"},
            {"field": "PCI"},
            {"field": "RNTI"}
        ],
        rowData=[]
    ),
    dcc.Graph(id="live-graph"),
    dcc.Interval(id="interval", interval=1000, n_intervals=0),
])
```

Código 6: Creación de la estructura visual mediante la función `app.layout`.

La actualización de los datos en tiempo real se realiza mediante `@app.callback`, que es el sistema encargado de que cada vez que el cronómetro (`dcc.Interval`) se activa cada segundo como configuramos en el apartado anterior, toma todos los datos ya existentes (almacenados en el DataFrame `agg_df`) y los valores actuales (del diccionario `info_ues` y la lista `vector_list`) para así reconstruir la gráfica (`live-graph`) y la tabla (`live-adgrid`), y mandarla hacia el programa Dash.

```
@app.callback(
    Output("live-graph", "figure"),
    Output("live-adgrid", "rowData"),
    Input("interval", "n_intervals")
)
def update_data(n):
    # 1. Dibujar gráfica
    fig = go.Figure()
    for col in agg_df.columns:
        fig.add_trace(go.Scatter(
            y=agg_df[col].to_list(),
            mode='lines+markers',
            name=col
        ))

    fig.update_layout(
        xaxis_title="Segundos de simulación",
        yaxis_title="Bytes Totales Descargados",
        margin=dict(t=30, b=0, l=0, r=0)
    )

    # 2. Ensamblar datos de la tabla cruzando 'vector_list' y 'info_ues'
    ---
    row_data = []
    for i in range(n_ues):
        row_data.append({
            "UE": f"UE {i}",
            "Bytes": vector_list[i],
            "PLMN": info_ues[i]["PLMN"],
```

```
            "PCI": info_ues[i]["PCI"],
            "RNTI": info_ues[i]["RNTI"]
        })

    return fig, row_data
```

Código 7: Decorador `@app.callback` mediante el cual se consigue actualizar los datos en tiempo real.

La Figura 1 representa las distintas funciones de la interfaz *Dash* desarrollada. Se observa que cuando el proceso de lectura de datos (`log-sim.py`) finaliza, el sistema vuelve a procesarlos de forma cíclica para mantener una monitorización continua. En la representación se distinguen los 4 usuarios junto a los siguientes parámetros: la velocidad agregada en el último segundo, obtenida de la lista `vector_list`; sus indicadores de red PLMN, PCI y RNTI, que se actualizan mediante el diccionario `info_ues`; y el registro histórico del tráfico acumulado, cuyos datos se extraen del *DataFrame* `agg_df`.

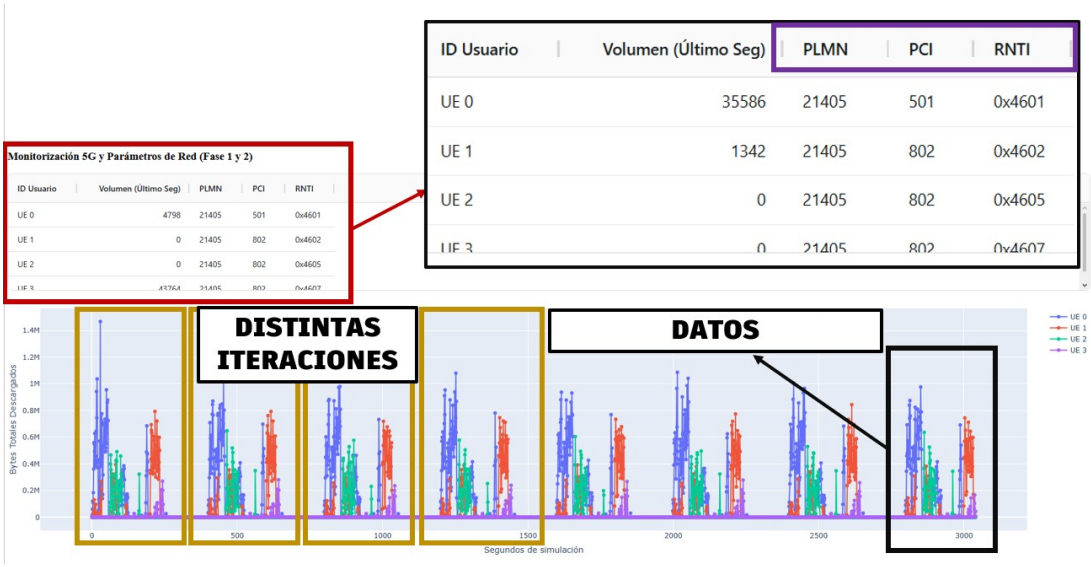

Figura 1: Ejemplo de una representación Dash realizada mediante el código.

2.5. Almacenamiento en archivo Parquet

El sistema implementa un mecanismo de guardado automático que guarda cada 30 segundos la evolución de los volúmenes agregados por UE en un archivo Parquet. Este proceso se realiza mediante una tarea asíncrona en segundo plano (`parquet_saver`), que utiliza un temporizador no bloqueante (`asyncio.sleep(30)`) para ejecutar el guardado sin interferir con el flujo principal de captura y procesamiento de datos. Antes de cada escritura, se verifica que el *DataFrame* contiene información (`if len(agg_df)>0`) y, en caso afirmativo, se vuelca a disco duro mediante el método optimizado `write_parquet()` de la librería Polars.

Para mantener un control de cuándo se realiza la actualización del archivo Parquet, se imprime por pantalla un mensaje que permite verificar visualmente que el hilo en segundo plano no se ha colgado y que los datos están siendo protegidos correctamente, mostrando el volumen temporal almacenado hasta el momento.

```
async def parquet_saver():
    archivo_parquet = "evolucion_volumenes.parquet"
    while True:
        await asyncio.sleep(30)
        if len(agg_df) > 0:
            agg_df.write_parquet(archivo_parquet)
            print(f"[BACKUP] Archivo actualizado: {len(agg_df)} segundos
    .")
```

Código 8: asyncio Saver.

A la hora de simular, se puede observar que existe una pequeña discrepancia entre el intervalo de guardado de 30 segundos y el número de filas registradas, lo cual responde a un fenómeno inherente a la concurrencia conocido como deriva temporal (*time drift*). Aunque el comando `await asyncio.sleep(30)` opera con precisión basándose en el reloj absoluto del sistema, el bucle de ingesta emplea un temporizador relativo que va acumulando poco a poco pequeñas demoras de procesamiento en cada ciclo. Estas pequeñas demoras se ven acentuadas por el bloqueo momentáneo del hilo principal durante las operaciones de Entrada/Salida (I/O) al escribir en el disco duro. Por tanto, esta oscilación es un comportamiento esperado, ya que el sistema prioriza la no interrupción del flujo asíncrono.

En la Figura 2, se observa el comportamiento que acabamos de describir, en el que la deriva temporal entre un instante de grabado y el siguiente no es exactamente de 30s. En el caso de la Figura 2 es de 28s.

```
[BACKUP] Archivo actualizado: 167 segundos.
```

(a) Captura del log indicando 167 segundos de datos.

```
127.0.0.1 - - [24/Apr/2026 15:52:59] "POST /_dash-update-component HTTP/1.1"
 200 -
[BACKUP] Archivo actualizado: 195 segundos.
127.0.0.1 - - [24/Apr/2026 15:53:00] "POST /_dash-update-component HTTP/1.1"
 200 -
127.0.0.1 - - [24/Apr/2026 15:53:01] "POST /_dash-update-component HTTP/1.1"
 200 -
```

(b) Captura del log indicando 195 segundos de datos.

Figura 2: Evidencia del proceso de guardado automático.

Para poder visualizar los resultados almacenados en el archivo Parquet, se ha desarrollado un sencillo script en Python, llamado `leer_parquet.py`, utilizando la librería Polars. Véase el Anexo A.2. En el código de este script, primero se configura la visualización para mostrar todas las filas sin límite, lo que permite inspeccionar completamente el

contenido del dataset. A continuación, se carga el archivo `evolucion_volumenes.parquet` en un DataFrame y, finalmente, se imprime en la terminal para su revisión directa. Aunque existen alternativas más avanzadas como el uso de herramientas de visualización, notebooks interactivos o consultas parciales para grandes volúmenes de datos, en este caso se ha optado por esta solución por su simplicidad y comodidad a la hora de revisar rápidamente toda la información.

En la Figura 3, se aprecia la salida por terminal que muestra un DataFrame de Polars cargado desde el archivo Parquet, con una forma de (4,146 filas, 4 columnas), detallando las columnas de volumen descargado para los cuatro usuarios del experimento (UE 0 a UE 3).

```
(ve-proyecto) PS E:\Universidad\5Curso\Laboratorio_Proyectos\Cuarto_Proyecto\LPIT-14253284\code> python leer_parquet.py
shape: (4_146, 4)
```

UE 0	UE 1	UE 2	UE 3
---	---	---	---
i64	i64	i64	i64
0	0	0	0
16900	0	0	0
59642	0	0	0
120572	0	0	0
108565	4121	0	0
183991	32032	0	0
360916	12997	0	0
559186	47652	0	0
549807	131339	0	0
492692	47008	0	0
627849	49551	0	0
399681	96751	0	0
426638	93629	0	0
379155	96857	0	0
457437	24215	0	0

Figura 3: Ejecución del script `leer_parquet.py` en el entorno virtual.

3. Fase 2

3.1. Identificación de PLMN, PCI y RNTI de cada UE

Con el objetivo de enriquecer el análisis del tráfico capturado en la Fase 1, se ha ampliado el sistema para incluir información de red asociada a cada equipo de usuario (UE). Esta mejora permite no solo monitorizar volúmenes de datos, sino también correlacionarlos con los parámetros clave del proceso de conexión en redes 5G.

3.1.1. Fundamentos de los Parámetros de Red

Para interpretar correctamente las métricas extraídas, es necesario definir los identificadores fundamentales utilizados en redes móviles:

- **PLMN (Public Land Mobile Network):** Identificador único del operador a nivel global. En el entorno de laboratorio, este valor permanece constante para todos los UEs.

- **PCI (Physical Cell Identity):** Identificador físico de la celda o sector de la estación base que atiende al usuario. Es esencial para distinguir la celda servidora de las vecinas y juega un papel clave en procesos como el *handover*.
- **RNTI (Radio Network Temporary Identifier):** Identificador temporal asignado dinámicamente por la red a cada UE durante una conexión activa. Permite direccionar de forma unívoca los mensajes en la interfaz radio.

3.1.2. Integración en el sistema y justificación

La incorporación de estos parámetros se ha realizado mediante una serie de modificaciones diseñadas para no interferir con el procesamiento en tiempo real del tráfico.

En primer lugar, se ha definido una expresión regular específica capaz de identificar en las trazas de señalización los valores de interés (PLMN, PCI y RNTI) junto con el identificador del usuario.

```
patron_info = re.compile(r"ue=(\d+).*?plmn=(\d+).*?pci=(\d+).*?rnti=(0x
    [0-9a-fA-F]+)")
```

Código 9: Expresión regular para la extracción de parámetros de red: PLMN, PCI y RNTI.

Dado que estos parámetros se asignan únicamente durante el establecimiento de la conexión y no cambian de forma continua, se ha optado por almacenarlos en un diccionario global en memoria. Esta estructura actúa como repositorio de estado, inicializando valores vacíos y actualizándolos en cuanto se detecta la información correspondiente.

```
info_ues = {i: {"PLMN": "-", "PCI": "-", "RNTI": "-"} for i in range(
    n_ues)}
```

Código 10: Inicialización del diccionario global para el almacenamiento de los parámetros de red por UE.

La lógica de extracción se integra dentro del productor asíncrono encargado de procesar el flujo de registros. De este modo, el sistema analiza cada línea en tiempo real y, cuando encuentra coincidencias, actualiza inmediatamente la información del UE correspondiente sin interrumpir la contabilización de tráfico.

```
match_info = patron_info.search(line)
if match_info:
    ue_id = int(match_info.group(1))
    if ue_id < n_ues:
        info_ues[ue_id]["PLMN"] = match_info.group(2)
        info_ues[ue_id]["PCI"]  = match_info.group(3)
        info_ues[ue_id]["RNTI"] = match_info.group(4)
```

Código 11: Lógica de extracción y actualización en tiempo real de los parámetros de red dentro del productor asíncrono.

Finalmente, esta información se integra en la interfaz gráfica mediante el *callback* de actualización. En este punto, se combinan los datos dinámicos de tráfico (volúmenes de bytes) con los parámetros de red almacenados, generando una vista unificada que permite analizar simultáneamente el comportamiento y el contexto de cada usuario.

```
    row_data = []
    for i in range(n_ues):
        row_data.append({
            "UE": f"UE {i}",
            "Bytes": vector_list[i],
            "PLMN": info_ues[i]["PLMN"],
            "PCI": info_ues[i]["PCI"],
            "RNTI": info_ues[i]["RNTI"]
        })
```

Código 12: Integración de los datos de tráfico y parámetros de red en la estructura de datos para su visualización en la interfaz Dash.

En conjunto, esta ampliación permite correlacionar en tiempo real el tráfico de cada UE con su información de conexión, aportando una visión más completa y útil para el análisis del sistema.

4. Main

La coordinación global del sistema se encuentra en la función principal `main`, diseñada como una rutina asíncrona encargada de gestionar el ciclo de vida de la aplicación y garantizar la ejecución concurrente de todos los módulos. Este enfoque permite integrar de forma eficiente la captura de datos, su procesamiento y la visualización en tiempo real sin interferencias entre componentes.

```
async def main():
    print("Iniciando...")
    asyncio.create_task(tail_file_producer(data_file, queue))
    asyncio.create_task(consumer(queue))
    asyncio.create_task(parquet_saver())

    await asyncio.to_thread(
        app.run,
        host="127.0.0.1",
        port=8051,
        debug=False,
    )

if __name__ == '__main__':
```

```
    asyncio.run(main())
```

Código 13: Estructura de main para la organización de tareas concurrentes y despliegue del servidor web.

Este diseño se apoya en tres pilares fundamentales:

- La instrucción `asyncio.run(main())`, la cual actúa como punto de entrada del sistema, inicializando el *event loop* de Python. Este bucle constituye el núcleo de la ejecución asíncrona, permitiendo gestionar múltiples tareas de forma cooperativa y eficiente sin necesidad de recurrir a múltiples procesos.
- El uso de `asyncio.create_task()`, se lanzan en segundo plano las tareas encargadas de la lectura continua del registro (*tail_file_producer*), el procesamiento de los datos (*consumer*) y el guardado periódico en disco (*parquet_saver*). Estas tareas se ejecutan de forma no bloqueante y comparten memoria, lo que permite mantener un flujo de datos continuo en tiempo real.
- La integración con la interfaz gráfica. El servidor web basado en Dash presenta un comportamiento síncrono y bloqueante. Para evitar que su ejecución interfiera con el bucle asíncrono principal, se recurre a `asyncio.to_thread()`, que delega su ejecución a un hilo independiente del sistema operativo. Esta separación es clave para garantizar que el servidor pueda atender peticiones de forma continua en el puerto que hemos escogido (en nuestro caso elegimos el 8051), mientras el sistema sigue capturando, procesando y almacenando datos de manera concurrente.

En conjunto, esta estrategia de organización permite desacoplar correctamente los distintos componentes del sistema, asegurando un buen funcionamiento para entornos de monitorización en tiempo real.

5. Funcionamiento del programa

En primer lugar, se debe iniciar el simulador de datos ejecutando el comando `python log-sim.py` en la terminal, tal y como se muestra en la Figura 4.

```
(ve-proyecto) PS E:\Universidad\5Curso\Laboratorio_Proyectos\Cuarto_Proyecto
\LPIT-14253284\Code> python log-sim.py
```

Figura 4: Ejecución del comando para iniciar el simulador de *logs*.

Este programa se encarga de leer un registro estático y volcarlo en tiempo real sobre un nuevo archivo (`cu-lan-ho-live.log`). Como se detalla en la Figura 5, una vez que el simulador termina de procesar los datos, entra en un bucle y vuelve a repetir el proceso (distintas iteraciones) para garantizar un flujo de información continuo e ininterrumpido.

Figura 5: Ejecución en bucle continuo del simulador.

Finalmente, en una segunda terminal, se ejecuta el programa principal mediante el comando `python log-parser_3.py`. La Figura 6 ilustra el resultado de este paso: el sistema levanta un servidor local donde se aloja la interfaz, comienza a detectar a los usuarios (UE) extrayendo sus parámetros de red, y registra las actualizaciones constantes (peticiones HTTP) que mantienen viva la pantalla de *Dash*.

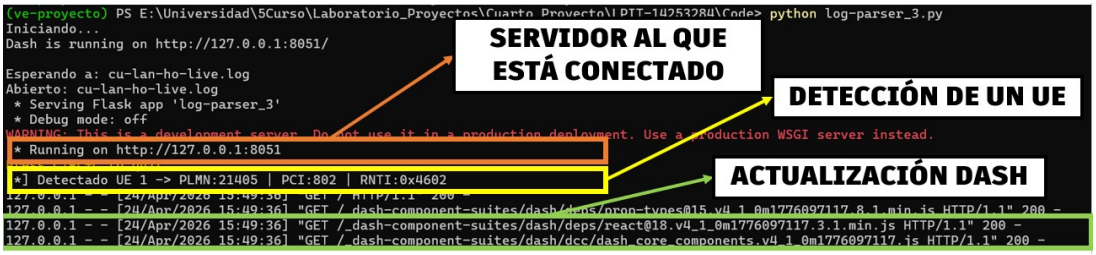

Figura 6: Inicialización del *parser*, servidor local y actualizaciones en *Dash*.

6. Conclusiones

En conclusión, el proyecto demuestra que la implementación de una arquitectura Productor-Consumidor basada en *asyncio* permite desacoplar eficazmente la captura y el procesamiento de datos, garantizando una monitorización fluida sin bloqueos. Asimismo, el uso de expresiones regulares optimiza el filtrado de grandes volúmenes de logs no estructurados, facilitando la extracción precisa de métricas clave de red. La adopción de *Polars* posibilita una gestión eficiente de datos de alto rendimiento, mientras que el

guardado regular de la información en formato *Parquet* asegura su integridad y disponibilidad para su posterior análisis. Además, la integración de *Dash* aporta una visualización interactiva que permite correlacionar el tráfico con el contexto de red de los usuarios monitorizados. Finalmente, la coordinación mediante una función principal asíncrona, junto con la ejecución del servidor web en un hilo independiente, garantiza un sistema robusto, capaz de operar de forma concurrente, estable y eficiente.

A. Código Fuente del Proyecto

En este anexo se recogen los scripts principales desarrollados en Python para cumplir con los objetivos de la Fase 1 y Fase 2 del proyecto.

A.1. Script principal de monitorización (log-parser.py)

```python
import asyncio
from pathlib import Path
import re
import time

import polars as pl
from dash import Dash, dcc, html, Input, Output
import plotly.graph_objects as go
import dash_ag_grid as dag

# ============================================================
# Links / Configuración
# ============================================================
version = "2.0 (Fase 1 + Fase 2)"
data_file = "cu-lan-ho-live.log"
n_ues = 4

# --- Patrones Regex ---
# Fase 1: Volumen DL
patron_sdap = re.compile(r"\[SDAP\s+\].*ue=(\d+).*DL: TX PDU.*pdu_len=(\d+)")
# Fase 2: Información de Red (Extrae UE, PLMN, PCI y RNTI)
patron_info = re.compile(r"ue=(\d+).*?plmn=(\d+).*?pci=(\d+).*?rnti=(0x[0-9a-fA-F]+)")

# ============================================================
# Shared asyncio queue & Data
# ============================================================
queue = asyncio.Queue()

agg_df = pl.DataFrame(
    schema={f"UE {i}": pl.Int64 for i in range(n_ues)}
)

vector_list = [0] * n_ues

# --- [NUEVO] Diccionario para guardar la info de red de cada UE ---
info_ues = {i: {"PLMN": "-", "PCI": "-", "RNTI": "-"} for i in range(n_ues)}
```

```python

# ============================================================
# asyncio producer:
# ============================================================
async def tail_file_producer(path: str, data_queue: asyncio.Queue):
    file = Path(path)
    print("Esperando a:", data_file)

    while not file.exists():
        await asyncio.sleep(0.5)

    with open(path, "r") as f:
        print("Abierto:", data_file)
        f.seek(0, 2)
        sumas_segundo = [0] * n_ues
        t_inicio = time.time()

        while True:
            line = f.readline()

            if line:
                # 1. Búsqueda de datos SDAP (Fase 1)
                match_sdap = patron_sdap.search(line)
                if match_sdap:
                    ue_id = int(match_sdap.group(1))
                    bytes_len = int(match_sdap.group(2))
                    if ue_id < n_ues:
                        sumas_segundo[ue_id] += bytes_len

                # 2.  Búsqueda de Información de Red (Fase 2) ---
                match_info = patron_info.search(line)
                if match_info:
                    ue_id = int(match_info.group(1))
                    if ue_id < n_ues:
                        # Guardamos los valores directamente en nuestro diccionario global
                        info_ues[ue_id]["PLMN"] = match_info.group(2)
                        info_ues[ue_id]["PCI"]  = match_info.group(3)
                        info_ues[ue_id]["RNTI"] = match_info.group(4)
                        print(f"[*] Detectado UE {ue_id} -> PLMN:{info_ues[ue_id]['PLMN']} | PCI:{info_ues[ue_id]['PCI']} | RNTI:{info_ues[ue_id]['RNTI']}")

            else:
                await asyncio.sleep(0.1)

            # --- CADA 1 SEGUNDO AGREGAMOS ---
```

```
            t_actual = time.time()
            if t_actual - t_inicio >= 1.0:
                vector = tuple(sumas_segundo)
                await data_queue.put(vector)
                sumas_segundo = [0] * n_ues
                t_inicio = t_actual

# ============================================================
# asyncio consumer:
# ============================================================
async def consumer(data_queue: asyncio.Queue):
    global agg_df, vector_list

    while True:
        vector_tuple = await data_queue.get()
        vector_list = list(vector_tuple)
        new_row = pl.DataFrame([vector_list], schema=agg_df.columns, orient="row")
        agg_df = pl.concat([agg_df, new_row], how="vertical")

        data_queue.task_done()

# ============================================================
# asyncio Saver
# ============================================================
async def parquet_saver():
    archivo_parquet = "evolucion_volumenes.parquet"
    while True:
        await asyncio.sleep(30)
        if len(agg_df) > 0:
            agg_df.write_parquet(archivo_parquet)
            print(f"[BACKUP] Archivo actualizado: {len(agg_df)} segundos.")

# ============================================================
# Dash app
# ============================================================
app = Dash(__name__)

app.layout = html.Div([
    html.H3("Monitorización 5G y Parámetros de Red (Fase 1 y 2)"),
    #  Grid Transpuesto para mostrar las propiedades limpiamente ---
    dag.AgGrid(
        id='live-adgrid',
        style={"height": 200, "width": "100%"}, # Altura aumentada
        columnSize='autoSize',
        columnDefs=[
```

```
            {"field": "UE", "headerName": "ID Usuario"},
            {"field": "Bytes", "headerName": "Volumen (Último Seg)", "type": "rightAligned"},
            {"field": "PLMN"},
            {"field": "PCI"},
            {"field": "RNTI"}
        ],
        rowData=[]
    ),
    dcc.Graph(id="live-graph"),
    dcc.Interval(id="interval", interval=1000, n_intervals=0),
])

@app.callback(
    Output("live-graph", "figure"),
    Output("live-adgrid", "rowData"),
    Input("interval", "n_intervals")
)
def update_data(n):
    # 1. Dibujar gráfica
    fig = go.Figure()
    for col in agg_df.columns:
        fig.add_trace(go.Scatter(
            y=agg_df[col].to_list(),
            mode='lines+markers',
            name=col
        ))

    fig.update_layout(
        xaxis_title="Segundos de simulación",
        yaxis_title="Bytes Totales Descargados",
        margin=dict(t=30, b=0, l=0, r=0)
    )

    # 2. Ensamblar datos de la tabla cruzando 'vector_list' y 'info_ues' ---
    row_data = []
    for i in range(n_ues):
        row_data.append({
            "UE": f"UE {i}",
            "Bytes": vector_list[i],
            "PLMN": info_ues[i]["PLMN"],
            "PCI": info_ues[i]["PCI"],
            "RNTI": info_ues[i]["RNTI"]
        })

    return fig, row_data
```

```python

# ============================================================
# Main
# ============================================================
async def main():
    print("Iniciando...")
    asyncio.create_task(tail_file_producer(data_file, queue))
    asyncio.create_task(consumer(queue))
    asyncio.create_task(parquet_saver())

    await asyncio.to_thread(
        app.run,
        host="127.0.0.1",
        port=8051,
        debug=False,
    )

if __name__ == '__main__':
    asyncio.run(main())
```

Código 14: Script log-parser.py completo.

A.2. Leer Parquet (leer_parquet.py)

```python
import polars as pl

# 1. Le decimos a Polars que el límite de filas es "infinito" (-1)
pl.Config.set_tbl_rows(-1)

# 2. Leemos el archivo
df_guardado = pl.read_parquet("evolucion_volumenes.parquet")

# 2. Imprimimos la tabla en la terminal
print(df_guardado)
```

Código 15: Script para leer Parquet.